

1. What is an Operating System?

An Operating System (OS) is system software that manages hardware resources and provides an interface between users and hardware.

2. What is a Kernel?

Kernel is the core of the OS that directly interacts with hardware and manages CPU, memory, processes, and devices.

3. Is Linux an Operating System or Kernel?

Linux is a kernel. When combined with tools and utilities (GNU), it becomes a complete OS.

4. Types of Kernels?

Monolithic
Microkernel
Hybrid
Exokernel
Nano-kernel]

5. What is a Shell?

Shell is a command-line interpreter that takes user commands and passes them to the kernel.

6. Shell vs Terminal?

Shell → Program that executes commands
Terminal → Interface used to access the shell

7. Popular Linux Distributions?

Ubuntu, CentOS, RHEL, Debian, Fedora, Amazon Linux

8. Linux vs Unix?

Linux is open-source; Unix is mostly proprietary.

9. What is GNU?

GNU is a collection of free software tools used with Linux to form a complete OS.

10. What is Open Source?

Software whose source code is freely available to use, modify, and distribute.

Linux File System Structure

11. What is / (root)?

Top-level directory of Linux file system.

12. /etc

Configuration files.

13. /bin

Essential user commands (ls, cp, mv).

14. /sbin

System administration commands.

15. /usr

User-installed applications and libraries.

16. /home

Home directories of users.

17. /tmp

Temporary files (auto-cleaned).

18. /var

Variable data (logs, mail, cache).

19. /opt

Optional third-party software.

20. /proc

Virtual filesystem with process and kernel info.

21. /dev

Device files (disks, USB, etc.).

22. /bin vs /usr/bin

/bin → essential commands

/usr/bin → non-essential user commands

Files & Permissions

23. File Permissions?

Read (r), Write (w), Execute (x)

24. What does rwx mean?

r → read

w → write

x → execute

25. chmod?

Changes file permissions.

26. chown?

Changes file ownership.

27. chmod vs chown?

chmod → permissions

chown → owner/group

28. umask?

Default permission setting for new files.

29. Hidden File?

File starting with dot (.bashrc).

30. How to check permissions?

ls -l

Basic Commands

31. pwd

Shows current directory.

32. cd

Change directory.

33. ls

List files and directories.

34. cp

Copy files/directories.

35. mv

Move or rename files.

36. rm

Delete files.

37. rm -r vs rm -f

-r → recursive

-f → force delete

38. touch

Create empty file.

39. mkdir

Create directory.

40. rmdir

Remove empty directory.

File Viewing & Editing

41. cat vs more?

cat → shows entire file

more → page by page

42. more vs less?

less allows backward navigation.

43. head

Shows first lines.

44. tail

Shows last lines.

45. tail -f

Live log monitoring.

46. vi / vim?

Text editor in Linux.

47. vi Modes?

Command

Insert

Escape

48. Search in vi?

/keyword

Searching & Filtering

49. grep?

Search text in files.

50. grep vs egrep?

egrep supports extended regex.

Extended regex support: Allows operators like +, ?, |, and () directly.

Faster pattern matching compared to basic grep in some cases.

Multiple patterns can be searched at once.

51. find?

Search files by name, size, type.

52. find vs locate?

locate is faster (uses database).

53. awk?

Text processing tool.

54. sed?

Stream editor for text replacement.

55. Pipe (|)?

Pass output of one command to another.

System & Hardware

56. uname -a?

Shows system information.

57. Disk space?

df -h

58. Disk usage?

`du -sh`

59. Memory usage?

`free -m`

`free -h`

60. CPU info?

`lscpu, top`

61. `top` vs `htop`?

`htop` is interactive and user-friendly.

62. Uptime?

`uptime`

63. Last reboot?

`last reboot`

64. Logged-in users?

`who, w`

65. `who` vs `whoami`?

`who` → logged users

`whoami` → current user

66. `man`?

Manual pages for commands.

Users & Groups

67. Root user?

Superuser with full permissions.

68. Root vs Normal user?

Root has unrestricted access.

69. Create user?

`useradd username`

70. Delete user?

`userdel username`

71. Switch user?

`su - username`

72. `sudo`?

Run commands with root privileges.

73. `/etc/passwd`?

User account information.

74. `/etc/shadow`?

Encrypted passwords.

Networking

75. IP address?

Unique identifier of a machine.

76. Internal vs External IP?

Internal → private network

External → internet-facing

77. `localhost`?

Local machine (127.0.0.1).

78. DNS?

Resolves domain name to IP.

79. Check IP?

ip addr, ifconfig

80. ping?

Check network connectivity.

81. netstat / ss?

View network connections.

82. Port?

Logical communication endpoint.

 Cloud / DevOps Basics

83. Instance?

Virtual machine in cloud.

84. Node?

Worker machine in cluster. VM + S/W application installed (docker)

85. VM vs Container?

VM runs full OS; answers container shares host OS.

86. SSH?

Secure remote login protocol.
Provides secure communication between two computers, replacing insecure protocols like Telnet and rlogin.

87. SCP?

Secure file copy between systems.

88. Environment variable?

Dynamic value used by processes.

89. PATH variable?

Defines executable search path.

90. Cron job?

Scheduled task in Linux.

Linux Commands & Monitoring (Real-Time Scenarios)

1. What is a zombie process?

A zombie process is a process that has completed execution but still has an entry in the process table because its parent has not read its exit status.

It does not consume CPU or memory, but it occupies a PID.

👉 Identified by Z state.

2. How will you find a specific process (like RCU) that is creating a problem?

Use process monitoring commands:

```
ps -ef | grep rcu
top
htop
```

Check CPU, memory usage, and process state.

RCU issues usually indicate kernel-level load or CPU contention.

3. What are 5 Linux monitoring commands?

Common production commands:

1. top – CPU & memory usage
2. htop – Interactive monitoring
3. ps – Process status
4. free -m – Memory usage
5. df -h – Disk usage

(Also acceptable: vmstat, iostat, uptime)

4. If a Java/Zoom app is consuming 85% CPU & memory, how will you fix it?

Step-by-step approach (important):

1. Identify process:

```
top
```

```
ps -ef | grep java
```

2. Check if load is expected (business hours?)

3. Restart application if stuck:

```
kill -9 <PID> # only if approved
```

4. Increase system resources if needed

5. Check logs for memory leaks

6. For Java: review JVM heap size

👉 Never kill blindly in production without approval

5. How do you check if your machine has been restarted?

```
uptime
last reboot
```

6. Command to check internet connectivity?

```
ping google.com
or
curl google.com
```

7. How can you search for specific files (.txt)?

```
find /path -name "*.txt"
or
locate *.txt
```

Shell Scripting

8. What is a shell script?

A shell script is a file containing multiple Linux commands executed sequentially by a shell.

9. Is a file without shebang a shell script?

Yes, but it must be executed using a shell explicitly:

```
sh script.sh
```

Without shebang, the system doesn't know which interpreter to use.

10. Purpose of shebang (#!/bin/bash)?

It tells the OS which interpreter should execute the script.

Permissions

11. Give only executable permission to owner

chmod 100 filename

or

chmod u+x,go-rwx filename

12. Full access to owner & group, no access to others

chmod 770 filename

13. Should you give 777 permission?

 No

Because it gives read, write, execute to everyone, causing serious security risks.

Used only for temporary testing, never in production.

14. Command to give executable permission

chmod +x filename

SSH Keys & Access Management

15. Which key do you share with manager – public or private?

 Public key only

 Never share private key

16. How do you generate SSH keys?

ssh-keygen

Creates:

id_rsa → Private key

[id_rsa.pub](#) → Public key

17. Use of private key?

Used for authentication when connecting to servers securely.

18. How do you connect using a private key?

ssh -i private_key.pem user@server_ip

19. Should you share private key?

 Never

Private key gives full server access and must remain confidential.

Memory & Disk Management

20. What is swap memory?

Swap memory is disk space used as virtual memory when RAM is full.

It prevents system crashes but is slower than RAM.

21. Check memory in human-readable format

free -h

22. Check disk space in human-readable format

df -h